

システム紹介

1. フロントのUI紹介
2. ファイル実行の進捗表示
3. 出力結果
4. 言語選択
5. 言語選択例
6. システムの動作レポート

フロントのUI紹介

検査状態

已载入模型:

qwen3-v1:4b

qwen3-v1:8b

qwen3-coder:480b-cloud

バス管理

動画フォルダパス

/mnt/f/上課影片

バス設定

画像フォルダパス

/app/data/images

バス設定

クイックパス

上課影片

講義圖片

フォルダツリー構造

/mnt/f/上課影片

_broken

週一

週四線上

動画処理

動画リストを再読み込み

動画総数 4 個

処理済み 2 個

保留中 2 個

一括処理 (2)

2025-09-29 13-31-03.mp4

保留中

2025-10-02 10-58-42.mp4

保留中

2025-10-02 10-58-42_TEST_MID30.mp4

完了

檔案: 2025-09-29 13-31-03.mp4

パス: 週一/2025-09-29 13-31-03.mp4

再生時間: 1小時 36分鐘 29秒

ファイルサイズ: 217 MB

処理開始

詳細

檔案: 2025-10-02 10-58-42.mp4

パス: 週一/2025-10-02 10-58-42.mp4

再生時間: 1小時 32分鐘 26秒

ファイルサイズ: 521 MB

処理開始

詳細

檔案: 2025-10-02 10-58-42_TEST_MID30.mp4

パス: 週一/2025-10-02 10-58-42_TEST_MID30.mp4

再生時間: 30分鐘 5秒

ファイルサイズ: 130 MB

結果を読み込み

再処理

詳細

test_5min.mp4

完了

檔案: test_5min.mp4

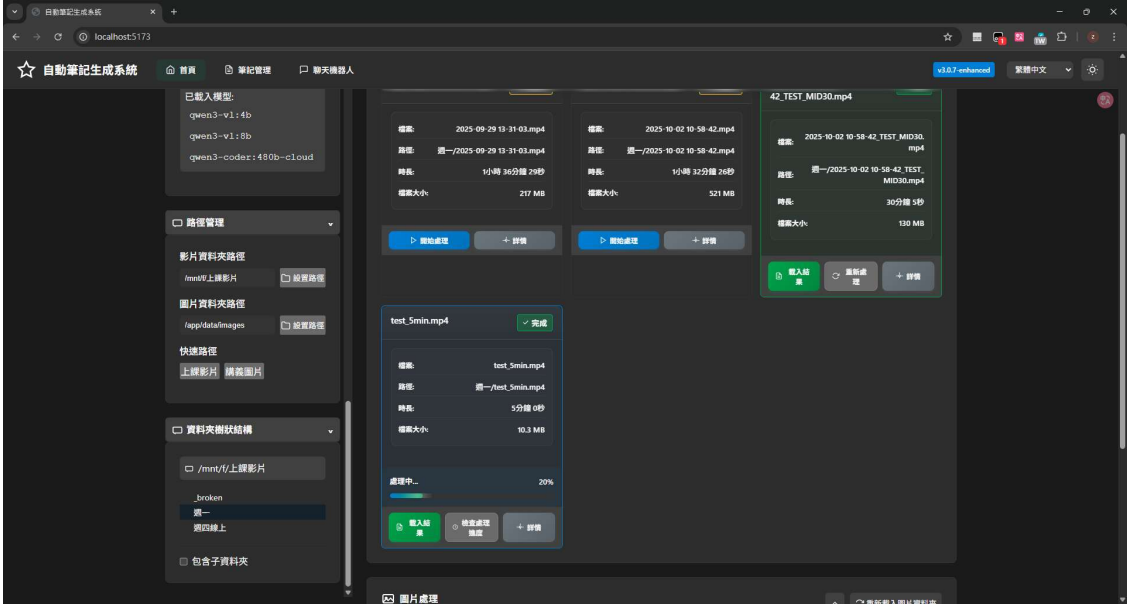
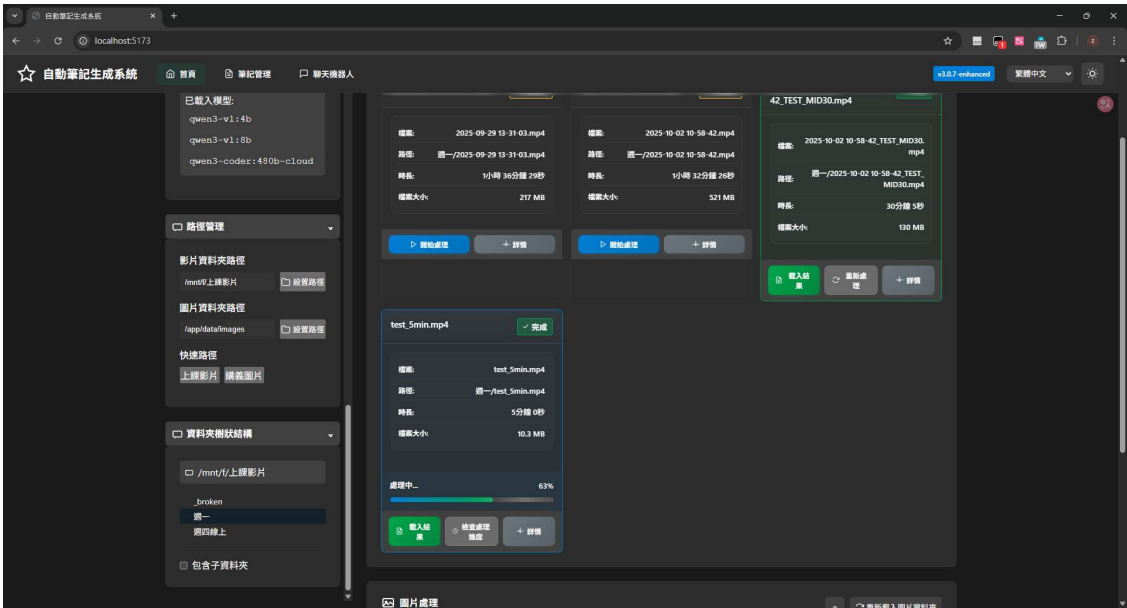
パス: 週一/test_5min.mp4

再生時間: 5分鐘 0秒

ファイルサイズ: 10.3 MB

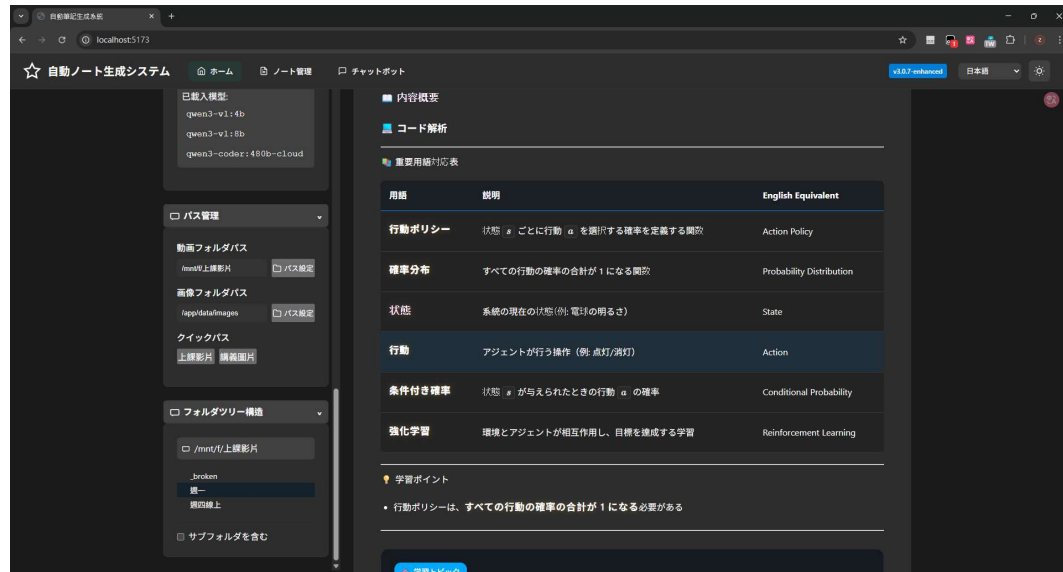
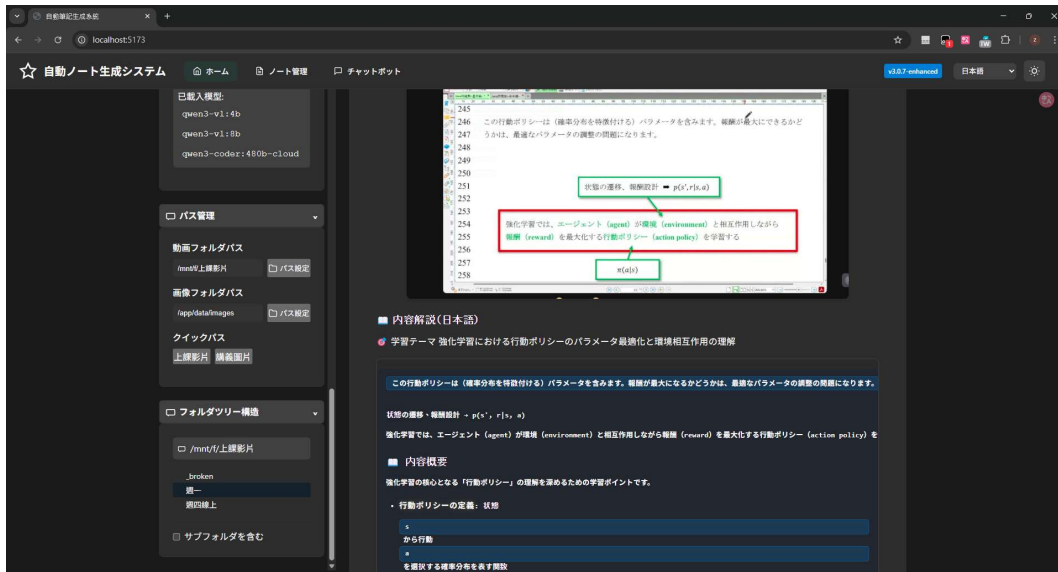
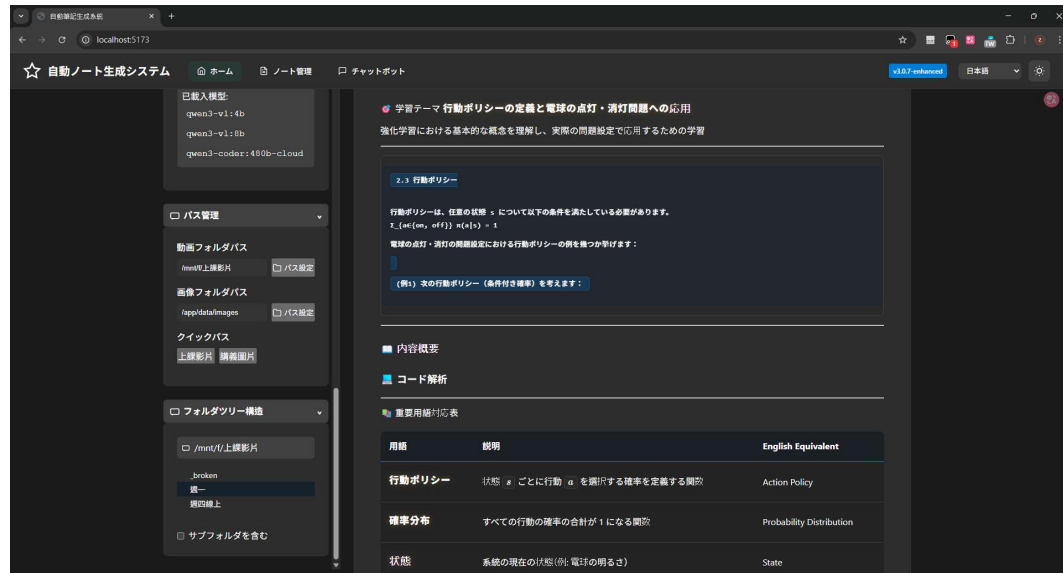
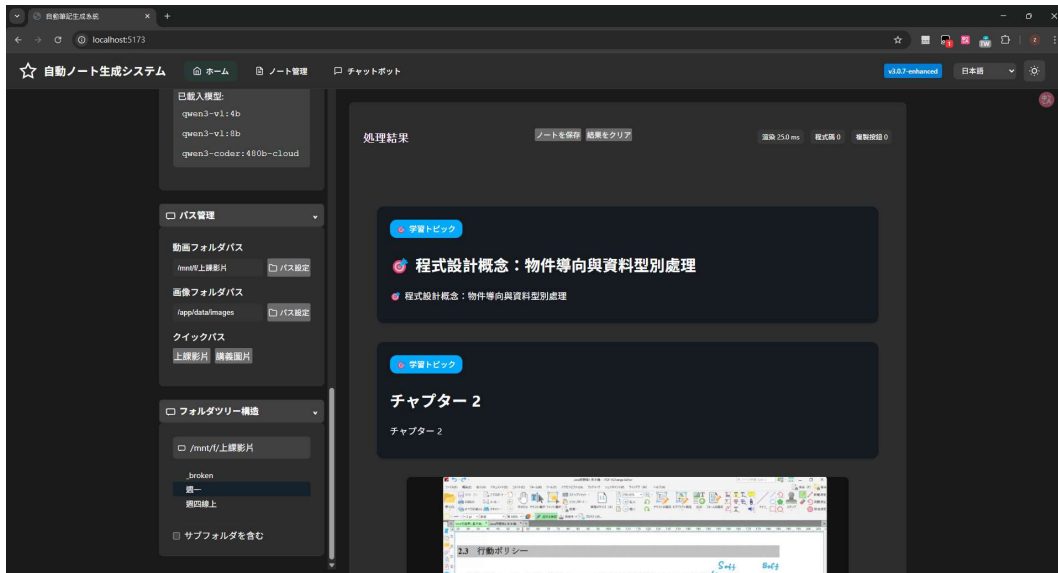
結果を読み込み

ファイル実行の進捗表示



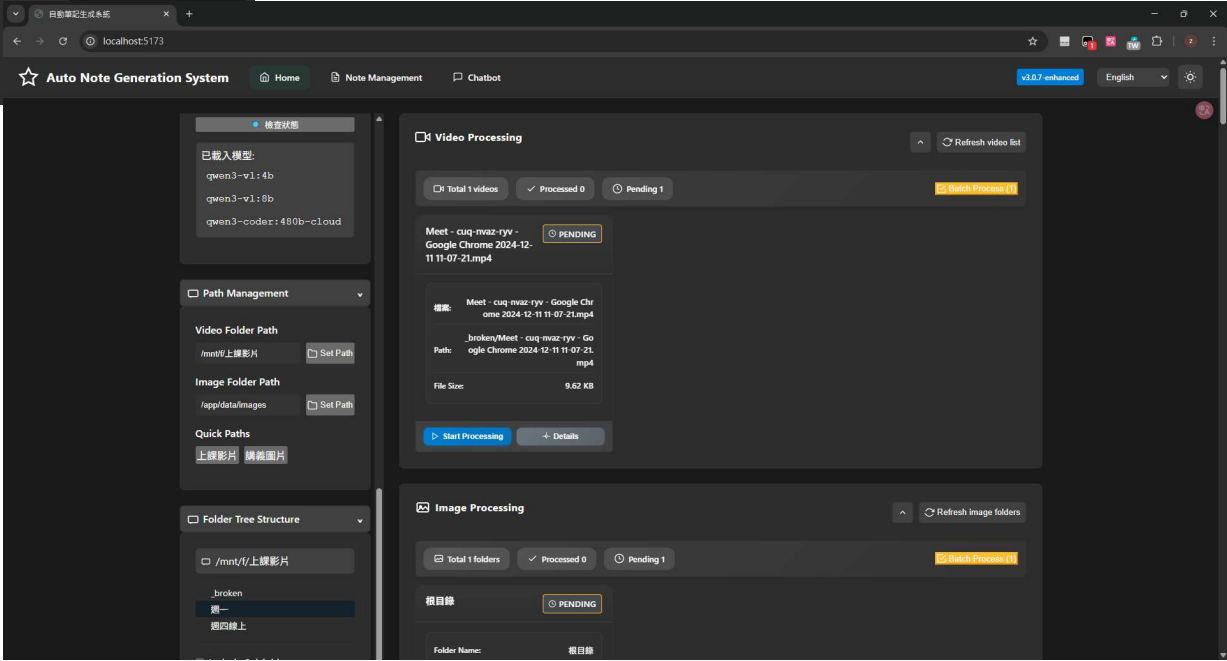
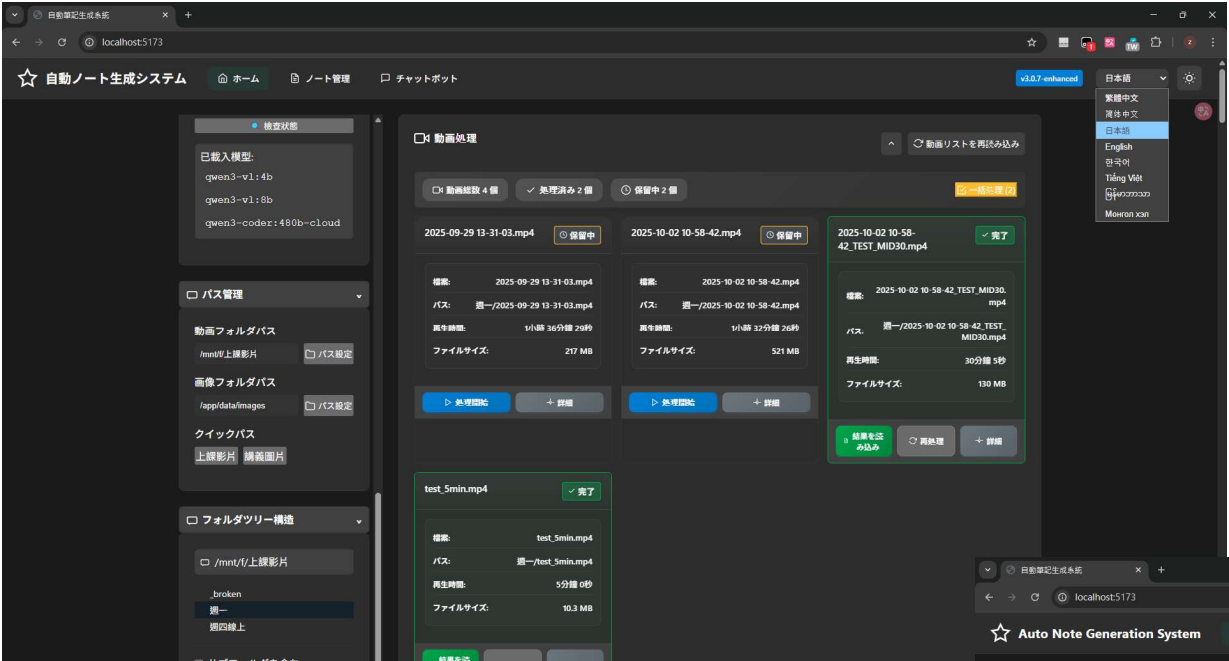
出力結果

今LLMで情報を出力できるけど、内容がまだ全部出ない時があって、UIも読みづらいから調整中



言語選択

言語選択ができて、フロントとLLMの出力もフロントの言語に合わせて自動で変わる



言語選択例

中国語UIに切り替えた時、日本語のポイントと中国語の説明を出せるようになった

自動筆記生成系統

localhost:5173

☆ 自動筆記生成系統

首頁筆記管理聊天機器人

v3.0.7-enhanced

繁體中文

已載入模型:

qwen3-v1:4b

qwen3-v1:8b

qwen3-coder:480b-cloud

路徑管理

影片資料夾路徑

/mnt/f/上課影片

設置路徑

圖片資料夾路徑

/app/data/images

設置路徑

快速路徑

上課影片

講義圖片

資料夾樹狀結構

/mnt/f/上課影片

_broken

週一

週四線上

包含子資料夾

昨今のサービスはネットワークを通して提供されるものがほとんど
→ 安定的なサービスの提供のためにサーバを安定的に運用する必要がある。

サーバ運用のために重要なポイント

- いつでも使うことができるか
- どこかで壊れても大丈夫か
- 自然災害で全部壊れても大丈夫か
- 性能不足にならないか
- 利用者が増えても大丈夫か
- 要件が頻繁に変わっても大丈夫か

重點說明（中文解釋）

學習主題 伺服器穩定運行與網路服務提供基礎概念分析

日文原文内容（非程式碼，避免被高亮） はじめに
昨今のサービスはネットワークを通じて提供されるものがほとんど → 安定的なサービスの提供のためにサーバを安定的に運用する必要がある。
サーバ運用のために重要なポイント

繁體中文解釋

內容概述

這張幻燈片是關於現代服務架構的基礎介紹。核心內容指出：當前大多數服務都是通過網路（ネットワーク）提供，因此要確保服務穩定性，必須穩定運行伺服器（サーバ）。這段文字強調了「網路服務依賴伺服器穩定性」的核心概念，並引出後續將探討伺服器運行中的關鍵點。

程式碼分析

此張幻燈片為概念性教學內容，無程式碼部分。但我們可以從技術角度分析相關概念：

- 「ネットワークを通して提供される」：表示服務通過網路協定（如HTTP/HTTPS）傳輸，與處理網路連線、流量控制等問題

システムの動作レポート

技術スタック

- フロントエンド: Vue 3, Vite, Element Plus, Pinia, Vue Router, Axios, Highlight.js/Prism, KaTeX
- バックエンド: FastAPI (Python 3.10), Uvicorn, PaddleOCR-VL, Torch, OpenCV, scenedetect
- LLM: Ollama (qwen3-vl:4b)
- デプロイ: Docker Compose (frontend, backend, ollama_local)

コアフロー

- 1. パス設定 (動画/画像)
- 2. 動画分割 + OCR/VLM
- 3. LLM で Markdown を生成
- 4. フロントで描画 (コードハイライト、KaTeX、学習ポイント)

主なコードファイル(フロントエンド)

- `frontend/src/main.js`
 - フロントエンドのエントリ。Vue/Router/Element Plus、ハイライト/KaTeXを読み込み、初期化。
- `frontend/src/views/Home.vue`
 - ホーム画面：処理カード、バッチ処理、統計バー；サイドバー折りたたみ（設定・パス・フォルダ・監視・Ollama 状態）。
- `frontend/src/components/LlmMarkdownNote.vue`
 - ノート描画：Markdown→HTML、コード/数式スタイル；空コード/重複ブロックを除去。
- `frontend/src/styles/note-compact.css`
 - ノートのレイアウトと配色：幅制限、余白、コード/数式/ボックスの統一

- frontend/src/components/SystemMetricsPanel.vue と src/composables/useSystemMetrics.js
- システム監視：5 秒ごとに /api/system/metrics をポーリングしリソース表示。
- ops/docker/docker-compose.yml
- コンテナ（frontend/backend/ollama）と環境変数/ボリュームを定義。
- ops/docker/Dockerfile
- CUDA ランタイム上に Python 依存、PaddleOCR、Torch を導入し、ヘルスチェック設定。

主なコードファイル(バックエンド)

- `backend/app/main.py`
 - FastAPI エントリ、ルーティング登録・起動設定。
- `backend/app/config.yaml`
 - バックエンド設定（モデル・パス・ポート・ログ等）。
- `backend/modules/api_routes.py`
 - API ルート：フロントからのリクエスト（動画/画像処理、ステータス、監視）
- `backend/modules/note_generator.py`
 - ノート生成の中核：シーン分割・OCR/VLM・LLM を統合

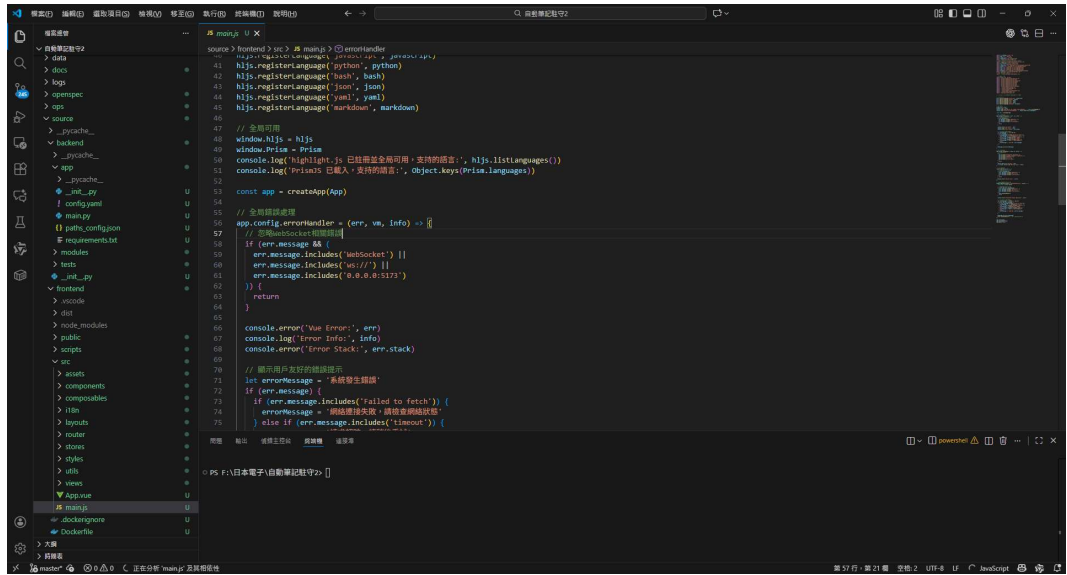
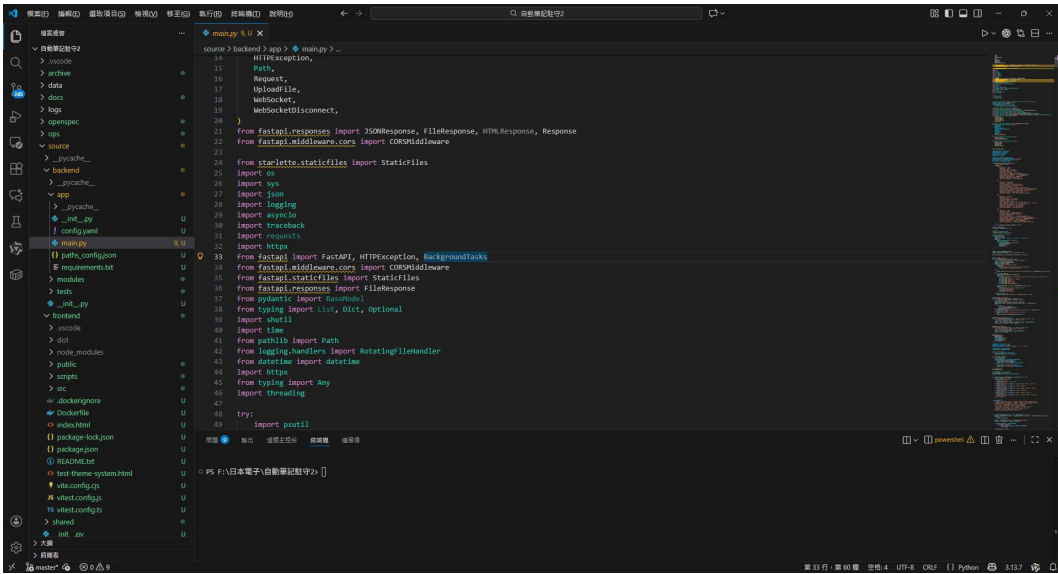
- backend/modules/summarize_video.py
- 動画パイプライン：シーン検出→フレーム抽出/前処理→OCR/VLM→ノート
- backend/modules/image_analyzer.py
- 画像前処理、OCR/VLM 解析
- backend/modules/llm_client.py
- Ollama 呼び出しのラッパー
- backend/modules/status_manager.py
- 進捗・ステータス管理、フロントがポーリング

処理フロー

- 1. フロント: 設定を受け取る (パス・モデル・パラメータ)
- 2. 動画: scenedetect でシーン分割 → OpenCV でフレーム抽出 → OCR/VLM → LLM で構造化段落生成
- 3. 画像: OpenCV による前処理 → OCR/VLM → LLM で構造化段落生成
- 4. 結果を返却: Markdown/JSON。進捗・メトリクスは API で参照

依存の用途

1. Torch：深層学習計算、GPU 加速の予備
2. OpenCV：画像前処理、フレーム操作
3. scenedetect：動画のシーン検出・分割
4. PaddleOCR-VL：OCR+ビジョン言語モデルで画像/フレームを解析
5. FastAPI：API フレームワーク；Uvicorn は ASGI サーバー



- 最初は音声＋講義スライドの両方で判断する計画だったけど、音声の収音があまり良くなくてうまくいかなかったから、最後はスライドだけで出力する方式にした。
- 前は Qwen2.5VL と PaddleOCR-4 を使ってたけど、バージョンアップ後、今は Qwen3VL 4B と PaddleOCR-VL に変えてシステムをちょっと最適化してる。速度も出るし、出力の精度もかなり良くなった。
- 今のところ、PaddleOCR-VL はやめようかなと思ってる。理由は Qwen3VL だけでももう十分レベル高いし、処理時間とリソースを節約したいから。
- UIはまだ改善中で、LLMが後端で作ったMarkdownをフロントでキレイに表示するのが結構むずい。どうやって見やすく整えるか、今いろいろ試してる。

以上です

作成者：林家誠